VULNERABILITY & CODE REVIEW REPORT

Project:	mx-chain-proxy-go
Domain:	RWA / DRWA / Core
Date:	2026-04-08
Reviewer:	Antigravity Code Analysis
Template:	8-Section PR Analysis Format
Total Issues:	10 (2 Critical · 5 Medium · 3 Informational)
Overall Status:	**REJECT — 8 issues require fixes before merge**

SECURITY & CODE REVIEW — PR ANALYSIS REPORT

Project: mx-chain-proxy-go

Reviewed: 2026-04-08

Reviewer: Antigravity Code Analysis

Total Issues Found: 10 (2 Critical · 5 Medium · 3 Informational)

ISSUE 1 — UNBUFFERED OS SIGNAL CHANNEL

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	Core / Process Lifecycle
File Changed	`cmd/proxy/main.go` — Lines 619–622
Change Type	Bug Fix

> **NOTE:**
> **"State"** = Any stored data that stays in the system (variables, DB records, or anything that survives a function call).

Does this change create new state, or update existing?

This is not a state-storing bug. The bug affects *runtime control flow* — specifically whether the process catches or silently drops the OS shutdown signal. No database or persistent state is involved.

---

2. Impacted System Flow

Field	Detail
Flow Name	Proxy Shutdown Flow
Entry Point	OS sends `SIGINT` (user presses Ctrl+C or `kill -2 <pid>`)
Exit Point	`closableComponents.Close()` + `httpServer.Shutdown(ctx)`

Short Description:

When the system operator wants to stop the proxy, the OS sends a `SIGINT` signal. The proxy is supposed to catch it on a channel, then close all background goroutines, drain in-flight HTTP requests, and exit cleanly. With an unbuffered channel, this handoff can be missed entirely.

> **NOTE:**
> **"Flow"** = The step-by-step path that data (or a signal) takes through the code.

Most complicated thing that happens:

The Go runtime receives the OS kernel signal and tries to push it into the `quit` channel. If the goroutine is not *already* blocked on `<-quit` at that exact nanosecond, the push is dropped — the runtime does not retry or queue it.

3. Invariant Analysis

Relevant System Invariant:

> *"The proxy must always shut down cleanly when the operator sends a stop command, closing all connections and releasing all resources."*

Field	Detail
Are invariants affected?	Yes
Correctness Rationale	With a buffered channel (`make(chan os.Signal, 1)`), the runtime stores the signal in the 1-slot buffer even if no receiver is waiting. The `<-quit` line will pick it up whenever it gets scheduled. This guarantees the invariant is always met.

---

4. Change Explanation

Behavior BEFORE:

```
quit := make(chan os.Signal)           // wrong-way zero-buffer — signal can be dropped
signal.Notify(quit, os.Interrupt, os.Kill)
<-quit
// Shutdown code...
```

If `SIGINT` arrives before `<-quit` is executing, the runtime silently drops it. The proxy ignores the stop command and keeps running.

Behavior AFTER:

```
quit := make(chan os.Signal, 1)        // correct-way 1-slot buffer — signal is always stored
signal.Notify(quit, os.Interrupt, syscall.SIGTERM)
<-quit
// Shutdown code always runs
```

The signal is buffered for up to 1 slot. Even if the receiver is not ready, the signal is saved and picked up when it is.

Rationale:

`go vet` flags this as a confirmed misuse pattern (`misuse of unbuffered os.Signal channel as argument to signal.Notify`). This is not theoretical — the Go standard library documentation explicitly requires a buffered channel.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Ctrl+C is pressed → proxy ignores it → operator must use `kill -9` (force kill)
Edge Cases	Signal arrives during early startup before `<-quit` is reached; signal arrives during heavy GC pause
Silent Failure Risk	**YES** — No error is logged. The proxy just keeps running as if nothing happened.
Risk Level	**Critical**

> **CAUTION:**
> **Silent Failure:** The operator sees no error, hears no alarm. They assume the proxy is shutting down but it is not. This can block deployments, cause port conflicts, and leave connections dangling.

---

6. Test Analysis

Field	Detail
Automated Coverage	`go vet ./...` — catches this automatically
Uncovered Areas	No unit test verifies graceful shutdown behavior end-to-end
Manual Verification	Run proxy, press Ctrl+C — observe whether it exits or hangs

How to prove the old behavior is gone:

1. Apply the fix.
2. Run `go vet ./cmd/proxy/...` — the warning disappears.
3. Start proxy, send `kill -SIGINT <pid>` — confirm clean exit in logs.

---

7. Cross-System Impact

Field	Detail
Downstream Affects	None — this only changes how the process handles OS signals
Dependency Assumptions	None — pure Go standard library change

Will other systems still work after deploy?

Yes. This change only affects the proxy's own shutdown behaviour. No API response format, no data schema, no downstream service is affected.

---

8. Final Decision

Field	Detail
Status	**REJECT — must fix before merge**
Reasoning	Confirmed `go vet` error. One line fix. Zero risk of regression. Blocking merge to protect operator ability to gracefully stop the service.

ISSUE 2 — MUTATING `HTTP.DEFAULTCLIENT` GLOBAL

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	Core / Network Communication
File Changed	`process/baseProcessor.go` — Lines 75–78
Change Type	Bug Fix

Does this create new state?

Yes — it mutates **global shared state**. `http.DefaultClient` is a process-wide singleton object. Mutating its `Timeout` field is permanently altering the global HTTP configuration for the entire proxy process.

---

2. Impacted System Flow

Field	Detail
Flow Name	Observer Node HTTP Communication
Entry Point	`NewBaseProcessor(requestTimeoutSec, ...)` called at startup
Exit Point	Every `CallGetRestEndPoint` / `CallPostRestEndPoint` invocation throughout proxy lifetime

Short Description:

At startup, the proxy creates a `BaseProcessor` which is supposed to get its own HTTP client with a configured timeout. Instead it accidentally modifies the global client — affecting every HTTP call in the entire process.

Most complicated thing that happens:

`http.DefaultClient` is a Go pointer (`*http.Client`). Assigning it to a local variable copies the pointer address, not the object. Both the local variable and `http.DefaultClient` point to the same memory. Mutating `.Timeout` through either name modifies the one shared object.

---

3. Invariant Analysis

Relevant System Invariant:

> *"Each component must own and control only its own resources. No component may silently alter shared global state."*

Field	Detail
Are invariants affected?	Yes
Correctness Rationale	Fix: allocate a new `&http.Client{Timeout: ...}` so the processor owns its own client. The global `http.DefaultClient` is never touched.

---

4. Change Explanation

Behavior BEFORE:

```
httpClient := http.DefaultClient        // wrong-way pointer alias — same object in memory
mutHttpClient.Lock()
httpClient.Timeout = time.Duration(requestTimeoutSec) * time.Second  // mutates global!
mutHttpClient.Unlock()
```

All third-party libraries or future code using `http.DefaultClient` will silently inherit the proxy's configured timeout.

Behavior AFTER:

```
httpClient := &http.Client{             // correct-way brand new, independently owned client
    Timeout: time.Duration(requestTimeoutSec) * time.Second,
}
```

The global is never touched. The processor owns exactly one private client.

Rationale:

The Go standard library documentation explicitly states: *"The DefaultClient's Timeout should not be modified."* Modifying it causes hidden side effects that are nearly impossible to debug in production.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Third-party library or health-check code using `http.DefaultClient` gets an unexpected timeout value
Edge Cases	Timeout set to 0 disables timeout on ALL http calls globally; concurrent initialization race
Silent Failure Risk	**YES** — Requests from other code paths may silently time out faster or slower than intended with no log
Risk Level	**High**

> **WARNING:**
> The currently required `mutHttpClient` global mutex is a symptom, not a fix. It protects the write but does not protect the hundreds of read sites that use `http.DefaultClient` throughout the process without knowing about this mutex.

---

6. Test Analysis

Field	Detail
Automated Coverage	No existing test catches global mutation
Uncovered Areas	Integration tests that spin up multiple `BaseProcessor` instances could expose this
Manual Verification	Run two `NewBaseProcessor` calls with different timeout values — verify they do not override each other

---

7. Cross-System Impact

Field	Detail
Downstream Affects	Any library imported by the proxy that uses `http.DefaultClient` is silently affected today
Dependency Assumptions	After fix: none. Each processor independently controls its timeout.

---

8. Final Decision

Field	Detail
Status	**REJECT — must fix before merge**
Reasoning	Three-line fix. Eliminates global side-effect. Also allows removing the now-unnecessary `mutHttpClient` global mutex.

ISSUE 3 — DRWA DENIALMESSAGE LOSES ORIGINAL CONTEXT

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	RWA / DRWA Compliance Signals
File Changed	`process/drwa_materializer.go` — Line 124
Change Type	Bug Fix

> **NOTE:**
> **New State Created?** The `DrwaDetails` struct (`data/drwa.go`) is populated per request and returned in the API response. It is not stored persistently, but it is the compliance signal delivered to clients.

---

2. Impacted System Flow

Field	Detail
Flow Name	DRWA Compliance Signal Extraction — Event Topic Path
Entry Point	`SimulateTransaction` API call that triggers a DRWA-aware SCR with event logs
Exit Point	`DrwaDetails.DenialMessage` field in the JSON response payload

Short Description:
When a DRWA contract emits a denial event, its topics carry both a structured code (e.g. `DRWA_KYC_REQUIRED`) and potentially additional human-readable context (e.g. holder address, reason phrase). The materializer extracts the code correctly but discards the surrounding context when setting `DenialMessage`.

Most complicated thing that happens:
The regex `\bDRWA_[A-Z0-9_]+\b` scans the full topic byte string and extracts only the matching code token. The surrounding context bytes (everything before and after the code) are thrown away, and `DenialMessage` is set to just the extracted code — identical to `DenialCode`.

---

3. Invariant Analysis

Relevant System Invariant:

> *"`DenialMessage` must always carry more human-readable context than `DenialCode`. They must never be equal."*

Field	Detail
Are invariants affected?	Yes — invariant is violated on the topic path
Correctness Rationale	Fix: set `result.DenialMessage = string(topic)` (full raw topic) instead of `denialIdentifier` (extracted code only). This matches what the fail-reason path already does correctly on line 71.

---

4. Change Explanation

Behavior BEFORE:

```
// Line 123-124 — topic path (WRONG)
result.DenialCode    = denialIdentifier   // e.g. "DRWA_KYC_REQUIRED"
result.DenialMessage = denialIdentifier   // wrong-way same value — context is lost
```

API response looks like:

```
{ "denialCode": "DRWA_KYC_REQUIRED", "denialMessage": "DRWA_KYC_REQUIRED" }
```

Behavior AFTER:

```
// Fixed topic path
result.DenialCode    = denialIdentifier   // "DRWA_KYC_REQUIRED"
result.DenialMessage = string(topic)      // correct-way "DRWA_KYC_REQUIRED holder erd1xyz missing
attestation level 3"
```

API response looks like:

```
{ "denialCode": "DRWA_KYC_REQUIRED", "denialMessage": "DRWA_KYC_REQUIRED holder erd1xyz missing
attestation level 3" }
```

Rationale:

The fail-reason path (line 71) already does this correctly. This is an inconsistency bug where the topic path forgot to use the full original string.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Compliance dashboards consuming `denialMessage` get no actionable information — same as reading `denialCode`
Edge Cases	Topic that is only the code and nothing else (message = code = OK); topic with binary data (hex-encoded separately in `DenialTopics`)
Silent Failure Risk	**YES** — The API returns valid JSON with no error. Nobody knows the message field is wrong unless they notice both fields are identical.
Risk Level	**Medium**

> **IMPORTANT:**
> For a compliance system (DRWA = regulatory workflow), losing denial context is a legal data quality risk, not just a cosmetic bug. Auditors may rely on `denialMessage` for human-readable denial records.

---

6. Test Analysis

Field	Detail
Automated Coverage	`TestMaterializeDRWADetailsExtractsDenialFromTopic` exists but only checks `DenialCode`, not `DenialMessage`
Uncovered Areas	No test asserts `DenialMessage != DenialCode` on the topic path
Manual Verification	Simulate a DRWA transfer denial with a rich topic payload and inspect JSON response

How to prove the old behavior is gone:

Add assertion to the existing test:

```
require.NotEqual(t, result.DenialCode, result.DenialMessage)
require.Contains(t, result.DenialMessage, "DRWA_GLOBAL_PAUSE")   // contains code
require.Greater(t, len(result.DenialMessage), len(result.DenialCode)) // but longer
```

---

7. Cross-System Impact

Field	Detail
Downstream Affects	Any client (mobile app, compliance dashboard, audit log) consuming the `drwa.denialMessage` JSON field
Dependency Assumptions	Consumer must be able to handle a longer string in `denialMessage` — no format change, just more data

---

8. Final Decision

Field	Detail
Status	**REJECT — fix required**
Reasoning	One-line fix. High correctness impact for a compliance-critical field. Must also add a test assertion to prevent regression.

ISSUE 4 — DEAD CONSTANT `DRWADENIALPREFIX` (DRWA)

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	RWA / DRWA Compliance
File Changed	`process/drwa_materializer.go` — Line 36
Change Type	Cleanup

Does this create new state? No. It is a declared but completely unused constant.

---

2. Impacted System Flow

Field	Detail
Flow Name	DRWA Denial Code Matching
Entry Point	`extractDRWADenialCode(message string)`
Exit Point	Same function — only `drwaDenialPattern` (the regex) is used

Short Description:
One constant (`drwaDenialPrefix = "DRWA_"`) and one compiled regex (`drwaDenialPattern`) live side by side. The constant does nothing. The regex does everything. The constant's presence creates an implied alternative implementation path that does not exist.

---

3. Invariant Analysis

Relevant System Invariant:

> *"Denial code matching must use word-boundary regex, never prefix substring matching, to prevent false positives from tokens like `XDRWA_...`."*

Field	Detail
Are invariants affected?	Yes — future developer might break the invariant using the dead constant
Correctness Rationale	Removing the constant eliminates the only available tool for someone to make the mistake. The regex is the sole implementation path.

---

4. Change Explanation

Behavior BEFORE:

```
const drwaDenialPrefix = "DRWA_"                          // wrong-way declared, never used
var drwaDenialPattern = regexp.MustCompile(`\bDRWA_[A-Z0-9_]+\b`)  // actually used
```

A future developer adds a quick check using `strings.HasPrefix(msg, drwaDenialPrefix)`. This silently matches `XDRWA_KYC_REQUIRED` as a valid denial code — a false positive compliance signal.

Behavior AFTER:

```
// drwaDenialPrefix removed entirely
var drwaDenialPattern = regexp.MustCompile(`\bDRWA_[A-Z0-9_]+\b`)  // one correct path
```

There is no tempting shortcut. Any future developer must use the regex.

Rationale:

The code comment on lines 13-16 explicitly warns against prefix matching. Having the prefix constant contradict that warning is a maintenance trap.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Future code uses the prefix constant for `strings.HasPrefix` → false positive DRWA signals
Edge Cases	Any event identifier or return message that starts with `DRWA_` but is not a canonical denial
Silent Failure Risk	**YES** — False positive compliance signals produce no error; wrong data is returned as if correct
Risk Level	**Medium (future risk)**

> **WARNING:**
> In a regulatory compliance context, a false positive DRWA signal (reporting a transfer was denied for compliance reasons when it was not) is a legal data integrity issue.

---

6. Test Analysis

Field	Detail
Automated Coverage	`TestMaterializeDRWADetailsIgnoresSubstringFalsePositive` covers `XDRWA_` prefix case — good
Uncovered Areas	No test verifies the constant is not used (compilers don't require constants to be used in Go)
Manual Verification	Delete the constant, run full test suite — must pass with zero changes

---

7. Cross-System Impact

Field	Detail
Downstream Affects	None — removing a dead constant has zero runtime effect
Dependency Assumptions	None

---

8. Final Decision

Field	Detail
Status	**REJECT — cleanup required before merge**
Reasoning	One-line delete. Zero regression risk. Eliminates a real future false-positive vulnerability in a compliance-critical path.

ISSUE 5 — RATE LIMITER MISSING `C.NEXT()`

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	Core / API Security / Rate Limiting
File Changed	`api/middleware/rateLimiter.go` — Lines 36–57
Change Type	Bug Fix

Does this create new state? No. The rate limiter `requestsMap` already exists. This is a control-flow bug in how the middleware passes (or fails to pass) execution to the next handler.

---

2. Impacted System Flow

Field	Detail
Flow Name	HTTP Request → Rate Limit Check → Route Handler
Entry Point	Every incoming HTTP request to a rate-limited endpoint
Exit Point	Actual route handler function (or `AbortWithStatusJSON` if limited)

Short Description:
In Gin, middleware handlers must call `c.Next()` to pass control to the next handler in the chain. When the request is within the rate limit, the current code falls off the end of the function without calling `c.Next()`. Gin handles this semi-transparently, but it creates fragile, non-idiomatic control flow.

---

3. Invariant Analysis

Relevant System Invariant:

> *"A middleware that does not abort must always call `c.Next()` to guarantee deterministic request propagation through the handler chain."*

Field	Detail
Are invariants affected?	Yes
Correctness Rationale	Add explicit `c.Next()` on both the "endpoint not limited" path and the "request within limit" path. Add `return` after `AbortWithStatusJSON` to prevent fall-through.

---

4. Change Explanation

Behavior BEFORE:

```
return func(c *gin.Context) {
    if !isEndpointLimited {
        return          // wrong-way no c.Next()
    }
    if numRequests >= limitForEndpoint {
        c.AbortWithStatusJSON(...)
    }
    // wrong-way falls off end — no c.Next()
}
```

Behavior AFTER:

```
return func(c *gin.Context) {
    if !isEndpointLimited {
        c.Next()        // correct-way explicit pass
        return
    }
    if numRequests >= limitForEndpoint {
        c.AbortWithStatusJSON(...)
        return          // correct-way no fall-through after abort
    }
    c.Next()            // correct-way within limit — proceed
}
```

Rationale:

Idiomatic Gin requires explicit `c.Next()`. Implicit chain advancement creates bugs when middleware is reordered or when new middleware is inserted.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Future middleware added after this one may not execute on some request paths
Edge Cases	Middleware reordering during refactor silently breaks request flow
Silent Failure Risk	**YES** — Requests may be processed incorrectly with no visible error
Risk Level	**Medium**

---

6. Test Analysis

Field	Detail
Automated Coverage	`rateLimiter_test.go` exists but tests only the counter logic, not the Gin chain propagation
Uncovered Areas	End-to-end middleware chain test verifying `c.Next()` is called
Manual Verification	Register two middleware handlers; verify the second always fires when rate limit is not hit

---

7. Cross-System Impact

Field	Detail
Downstream Affects	Any middleware registered after this one — behavior depends on whether `c.Next()` is called
Dependency Assumptions	None

---

8. Final Decision

Field	Detail
Status	**REJECT — fix required**
Reasoning	Two-line fix. Makes control flow explicit and prevents silent middleware chain bugs.

ISSUE 6 — UNBOUNDED METRIC KEYS FROM ON-CHAIN DATA (DRWA)

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	RWA / DRWA Observability
File Changed	`process/drwa_materializer.go` — Lines 67–68, 84–85, 121–122
Change Type	Bug Fix / Security

> **NOTE:**
> **New State Created?** Yes. Every unique metric key creates a new entry in the in-memory `map[string]uint64` inside `drwaProcessObservability`. This map is **never garbage-collected** between reset cycles.

---

2. Impacted System Flow

Field	Detail
Flow Name	DRWA Denial Detection → In-Memory Metric Recording
Entry Point	`SimulateTransaction` call → `materializeDRWADetails`
Exit Point	`proxyDRWAMetrics.counters` map (grows in memory)

Short Description:
Each DRWA denial detection records a metric keyed by the denial code (e.g. `drwa_denial_code_drwa_kyc_required`). Because the key comes from on-chain data (attacker-controlled), a malicious contract can generate thousands of unique long codes, each creating a new entry in the map — causing memory to grow without bound.

Most complicated thing that happens:
Each call to `recordProxyDRWAMetric("drwa_denial_code_" + strings.ToLower(denialIdentifier))` acquires a mutex, creates a new `map[string]uint64` entry if the key is new, and releases the mutex. The regex `\bDRWA_[A-Z0-9_]+\b` enforces charset but not length. `denialIdentifier` can be up to 500+ characters long.

---

3. Invariant Analysis

Relevant System Invariant:

> *"Memory used by the proxy must be bounded by configuration, not by the content of on-chain transactions."*

Field	Detail
Are invariants affected?	Yes — metric map grows proportionally to attacker input
Correctness Rationale	Cap the metric label to 64 characters before use as a map key. This limits the number of distinct entries to `36^64` (effectively infinite variety but each key is bounded in size).

---

4. Change Explanation

Behavior BEFORE:

```
// Line 68 — unbounded attacker-controlled key
recordProxyDRWAMetric("drwa_denial_code_" + strings.ToLower(denialIdentifier))
// denialIdentifier can be any length matching \bDRWA_[A-Z0-9_]+\b
```

Attack scenario:

- Adversary deploys a DRWA-function contract that emits `drwaTransferDenied` events.
- Each event has a topic like `DRWA_AAAAAAA...AAA` (unique 500-char code per tx).
- Proxy processes simulate calls → each creates a new map entry → memory grows.

Behavior AFTER:

```
func safeDenialMetricLabel(id string) string {
    const max = 64
    id = strings.ToLower(id)
    if len(id) > max { id = id[:max] }
    return id
}

recordProxyDRWAMetric("drwa_denial_code_" + safeDenialMetricLabel(denialIdentifier))
```

Keys are always ≤ 64 characters. Map cardinality is bounded.

Rationale:

Metric label cardinality attacks are a well-known class of vulnerability in observability systems (Prometheus cardinality explosion). The fix here is standard — truncate user-controlled labels before using them as map keys.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Proxy OOM (Out of Memory) crash after processing many simulate-tx calls with unique adversarial denial codes
Edge Cases	Very short codes (fine); codes of exactly 64 chars (fine); codes longer than 64 (truncated safely)
Silent Failure Risk	**YES** — Memory grows quietly. No alert until OOM crash. Crash presents as unexplained proxy restart.
Risk Level	**Medium** (requires significant spam volume; but `simulate` endpoint has no cost, making it accessible)

> **CAUTION:**
> The `simulate` endpoint requires no gas and no fee from the caller. An attacker can spam it at near-zero cost, making this memory growth vector accessible to anyone with network access to the proxy.

---

6. Test Analysis

Field	Detail
Automated Coverage	`drwa_materializer_test.go` — no test for metric key length
Uncovered Areas	Metric cardinality under adversarial input; map size under sustained spam
Manual Verification	Run the benchmark `BenchmarkMaterializeDRWADetails_FromFailReason` with a 500-char denial code; measure heap growth

---

7. Cross-System Impact

Field	Detail
Downstream Affects	None on API response format. Metric keys are never serialised to clients.
Dependency Assumptions	None

---

8. Final Decision

Field	Detail
Status	**REJECT — fix required**
Reasoning	5-line helper function. Protects against a real memory growth attack vector on the simulate endpoint. Must apply at all 3 call sites in the materializer.

ISSUE 7 — `OS.KILL` (SIGKILL) IS UNCATCHABLE

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	Core / Process Lifecycle
File Changed	`cmd/proxy/main.go` — Line 621
Change Type	Bug Fix / Cleanup

Does this create new state? No. This is dead code — the signal registration is silent no-op.

---

2. Impacted System Flow

Field	Detail
Flow Name	Proxy Shutdown Flow (system-initiated)
Entry Point	`systemctl stop proxy` / `docker stop` / Kubernetes pod eviction → SIGTERM
Exit Point	`closableComponents.Close()` + `httpServer.Shutdown()`

Short Description:
Infrastructure management tools (systemd, Kubernetes, Docker) send `SIGTERM` to politely ask a process to shut down. If it doesn't respond, they escalate to `SIGKILL` after a timeout. The current code registers for `os.Kill` (SIGKILL) — which is uncatchable — while not registering for `SIGTERM` — which is the correct signal to catch. This means infrastructure-initiated shutdowns silently bypass the graceful shutdown logic.

---

3. Invariant Analysis

Relevant System Invariant:

> *"The proxy must perform a graceful shutdown on all standard system stop requests, including infrastructure tools."*

Field	Detail
Are invariants affected?	Yes — SIGTERM from systemd/K8s/Docker never triggers graceful shutdown
Correctness Rationale	Replace `os.Kill` with `syscall.SIGTERM`. Now all three common stop paths (Ctrl+C = SIGINT, system stop = SIGTERM) trigger the shutdown handler.

---

4. Change Explanation

Behavior BEFORE:

```
signal.Notify(quit, os.Interrupt, os.Kill)
// os.Kill = SIGKILL → kernel kills process directly, never reaches channel
// SIGTERM (from docker stop / systemctl stop) → not registered → not caught
```

`docker stop` sends SIGTERM → proxy does not respond → Docker waits 10s → sends SIGKILL → proxy terminated with no graceful shutdown.

Behavior AFTER:

```
signal.Notify(quit, os.Interrupt, syscall.SIGTERM)
// SIGINT  (Ctrl+C)           → caught correct-way
// SIGTERM (infrastructure)   → caught correct-way
// SIGKILL (last resort kill) → uncatchable by design — removed from registration
```

Rationale:

Every container-based deployment in production receives SIGTERM during rolling updates, scaling events, and maintenance. Not handling it means every deploy silently force-kills the proxy.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	`docker stop proxy` / `systemctl stop proxy` → waits full timeout → force kills → connections dropped mid-flight
Edge Cases	Kubernetes pods with `terminationGracePeriodSeconds` > 10s — all wasted; proxy never gracefully drains
Silent Failure Risk	**YES** — In logs it looks like a clean exit (SIGKILL doesn't print an error). Only request-level logs reveal dropped connections.
Risk Level	**Medium**

---

6. Test Analysis

Field	Detail
Automated Coverage	`go vet` does not catch this specific pattern (only the buffered channel issue)
Uncovered Areas	No integration test sends SIGTERM to the proxy process
Manual Verification	Run proxy in Docker, `docker stop`, observe whether graceful shutdown log lines appear

---

7. Cross-System Impact

Field	Detail
Downstream Affects	CI/CD pipeline — rolling deployments drop in-flight requests until this is fixed
Dependency Assumptions	None — standard `syscall` package

---

8. Final Decision

Field	Detail
Status	**REJECT — fix required**
Reasoning	One-line change. Required for reliable operation in any containerised / orchestrated environment.

ISSUE 8 — DRWA METRICS NEVER EXPOSED AT RUNTIME

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	RWA / DRWA Observability
File Changed	`process/drwa_observability.go` (all), `process/mrv_observability.go` (all)
Change Type	Feature Gap

Does this create new state? No. The metric counters already exist and are already being incremented. The gap is that they are never *read* in production code.

---

2. Impacted System Flow

Field	Detail
Flow Name	DRWA Signal Detection → Observability → Operator Dashboard
Entry Point	`recordProxyDRWAMetric(...)` called during `materializeDRWADetails`
Exit Point	**Dead end** — metrics are incremented but never surfaced

Short Description:
The counters track how many DRWA compliance signals were accepted, rejected, and how many denials were detected. These counters are designed for operator visibility, but `snapshotProxyDRWAMetrics()` is called only in test files. No production code path reads or exposes these numbers.

---

3. Invariant Analysis

Relevant System Invariant:

> *"All observable compliance events must be surfaced to operators in real time."*

Field	Detail
Are invariants affected?	Yes — DRWA compliance events are observable internally but invisible externally
Correctness Rationale	Wire `snapshotProxyDRWAMetrics()` into an existing status endpoint or add a dedicated `/drwa/metrics` endpoint.

---

4. Change Explanation

Behavior BEFORE:

```go
// drwa_observability.go
func snapshotProxyDRWAMetrics() map[string]uint64 { ... }  // called only in test files
func resetProxyDRWAMetrics()                       { ... }  // called only in test files
```

Operator sees zero visibility into compliance events. Cannot alert on denial spikes.

Behavior AFTER (proposed):

```go
// In an appropriate status/metrics handler:
drwaMetrics := snapshotProxyDRWAMetrics()
for key, val := range drwaMetrics {
    statusMetricsHandler.SetUInt64Value("drwa_" + key, val)
}
```

Operator can query the status endpoint and see:

```
{ "drwa_signal_accepted": 142, "drwa_denial_detected": 7, "drwa_signal_rejected": 0 }
```

Rationale:

Building observability infrastructure and then not connecting it to any output is equivalent to having a smoke detector with no alarm speaker.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	DRWA denial spike goes undetected; no alerts fire; regulatory audit has no evidence
Edge Cases	Reset interval too short → counters cleared before snapshot; too long → stale data
Silent Failure Risk	**YES** — Everything silently works internally. Operators have no window into it.
Risk Level	**Low** (infrastructure works; only visibility is missing)

---

6. Test Analysis

Field	Detail
Automated Coverage	Metric increment/snapshot tested via unit tests
Uncovered Areas	No test verifies that metrics appear in any API endpoint
Manual Verification	After wiring: call `/node/status`, confirm DRWA metric keys are present in response

---

7. Cross-System Impact

Field	Detail
Downstream Affects	Monitoring dashboards / alerting rules need updating to consume new metric keys
Dependency Assumptions	Depends on which endpoint is chosen to expose the metrics

---

8. Final Decision

Field	Detail
Status	**HOLD — schedule in next sprint**
Reasoning	No runtime correctness impact. Medium effort to wire correctly. Should be tracked as a follow-up task with compliance team input on which metrics are required.

ISSUE 9 — BENCHMARK USES NON-CANONICAL EVENT IDENTIFIER (DRWA)

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	RWA / DRWA / Test Quality
File Changed	`process/drwa_materializer_benchmark_test.go` — Line 29
Change Type	Bug Fix (Test)

Does this create new state? No. This is a test file — affects benchmark accuracy only.

---

2. Impacted System Flow

Field	Detail
Flow Name	DRWA Materializer — Log-Events Scan Path
Entry Point	`BenchmarkMaterializeDRWADetails_FromSCRAndLogs`
Exit Point	Result of `materializeDRWADetails` call

Short Description:
The benchmark is designed to measure the cost of processing SCR results that contain DRWA event logs. However, because the event `Identifier` used (`"drwaDenial"`) is not in the canonical events whitelist, the entire log-events scan code path is skipped. The benchmark measures only the cheaper SCR return-message check, producing misleadingly fast results.

---

3. Invariant Analysis

Relevant System Invariant:

> *"Benchmarks must exercise the actual production code path they describe in their name."*

Field	Detail
Are invariants affected?	Yes — benchmark named `FromSCRAndLogs` does not exercise the Logs path
Correctness Rationale	Change identifier to a canonical one (e.g. `"drwaTransferDenied"`) so `isDRWACanonicalEvent` returns `true` and the event loop executes.

---

4. Change Explanation

Behavior BEFORE:

```
Identifier: "drwaDenial",          // wrong-way not in drwaCanonicalEvents → entire loop skipped
```

The `isDRWACanonicalEvent("drwaDenial")` check on line 107 returns `false`. The inner event loop body never executes. Benchmark result reflects only map iteration overhead, not real event processing.

Behavior AFTER:

```
Identifier: "drwaTransferDenied",  // correct-way in drwaCanonicalEvents → full path executed
```

The benchmark now exercises: canonical event check → trusted emitter check → topic iteration → denial code extraction → metric recording.

Rationale:

A benchmark that silently skips the expensive path will not catch performance regressions introduced in that path.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Performance regression introduced in the log-events path goes undetected
Edge Cases	None
Silent Failure Risk	**YES** — CI benchmark shows no regression because the broken path is never measured
Risk Level	**Low** (no production runtime impact)

---

6. Test Analysis

Field	Detail
Automated Coverage	Unit test `TestMaterializeDRWADetailsFromLogs` does use a correct identifier — only the benchmark is wrong
Uncovered Areas	Log-event scan path in benchmarks
Manual Verification	Run benchmark before and after fix — result should be notably slower after (more code actually runs)

---

7. Cross-System Impact

Field	Detail
Downstream Affects	None
Dependency Assumptions	None

---

8. Final Decision

Field	Detail
Status	**REJECT — one-line fix, low effort**
Reasoning	One-line change. Ensures benchmark actually guards the performance of the code it purports to measure.

ISSUE 10 — `SENDMULTIPLETRANSACTIONS` SILENT PARTIAL FAILURE

1. Change Scope

Field	Detail
Repository	mx-chain-proxy-go
Feature / Domain	Core / Transaction Submission
File Changed	`process/transactionProcessor.go` — Lines 304–363
Change Type	Bug Fix

> **NOTE:**
> **New State Created?** No new state. The bug is in the *reporting* of which transactions were successfully submitted — the blockchain state updates happen on the nodes, not in the proxy.

---

2. Impacted System Flow

Field	Detail
Flow Name	Batch Transaction Submission Flow
Entry Point	`POST /transaction/send-multiple` with a list of signed transactions
Exit Point	`MultipleTransactionsResponseData` with `NumOfTxs` and `TxsHashes`

Short Description:
The function groups transactions by target shard and then loops over shards, submitting each group. If any one shard's observer lookup fails mid-loop, the function immediately returns an error, completely abandoning the result data from previously-succeeded shards. The caller cannot distinguish "nothing was sent" from "some were sent, some failed".

Most complicated thing that happens:
Transactions from shards 0 and 1 are sent successfully (their hashes accumulated in `txsHashes`). On shard 2, `GetObservers` returns `ErrMissingObserver`. The function discards `txsHashes` entirely and returns `data.MultipleTransactionsResponseData{}` — an empty struct — alongside the error.

---

3. Invariant Analysis

Relevant System Invariant:

> *"The API response for a batch submission must always accurately reflect which transactions were accepted and which were not. It must never report zero successes if some transactions actually landed."*

Field	Detail
Are invariants affected?	Yes — invariant is violated on partial failure
Correctness Rationale	Use `continue` instead of `return` on per-shard errors. Log the shard-level failure. Return accumulated partial results alongside a list of shard-level errors.

---

4. Change Explanation

Behavior BEFORE:

```
for shardID, groupOfTxs := range txsByShardID {
    observersInShard, err := tp.proc.GetObservers(shardID, ...)
    if err != nil {
        return data.MultipleTransactionsResponseData{}, ErrMissingObserver  // wrong-way drops all
prior results
    }
    // ... send and accumulate txsHashes
}
return data.MultipleTransactionsResponseData{NumOfTxs: totalTxsSent, TxsHashes: txsHashes}, nil
```

Behavior AFTER:

```
var shardErrors []string
for shardID, groupOfTxs := range txsByShardID {
    observersInShard, err := tp.proc.GetObservers(shardID, ...)
    if err != nil {
        shardErrors = append(shardErrors, fmt.Sprintf("shard %d: %v", shardID, err))
        continue   // correct-way keep going — partial success is still success
    }
    // ... send and accumulate txsHashes
}
if len(shardErrors) > 0 {
    log.Warn("partial failure in batch send", "errors", shardErrors)
}
return data.MultipleTransactionsResponseData{NumOfTxs: totalTxsSent, TxsHashes: txsHashes}, nil
```

Rationale:

A client that gets `ErrMissingObserver` as a response will retry all transactions. Transactions that already landed will fail on retry with nonce errors. Transactions that did not land will succeed. The client has no way to know which is which — this is a correctness failure.

---

5. Failure & Risk Analysis

Field	Detail
Failure Mode	Client retries all transactions → duplicate nonce errors for already-sent txs → user confusion
Edge Cases	All shards fail (legitimate full failure); exactly one shard fails; zero transactions to send
Silent Failure Risk	**YES** — Client receives `ErrMissingObserver` with no data. Already-sent transactions are silently orphaned in the client's retry queue.
Risk Level	**Medium**

> **IMPORTANT:**
> This acknowledged with a `// TODO` comment in the existing code at line 308. It is a known, unresolved correctness issue.

---

6. Test Analysis

Field	Detail
Automated Coverage	`transactionProcessor_test.go` exists but does not test the partial failure path
Uncovered Areas	Test case: 3 shards, shard 2 returns `ErrMissingObserver` — assert that shard 0 and 1 hashes are in the response
Manual Verification	Run with a config that has no observer for one shard; submit cross-shard batch; inspect response

---

7. Cross-System Impact

Field	Detail
Downstream Affects	All clients using `/transaction/send-multiple` — wallets, relayers, batch submission services
Dependency Assumptions	API response format does not change — same struct, may now return partial data instead of empty data

---

8. Final Decision

Field	Detail
Status	**HOLD — schedule as tracked issue**
Reasoning	Medium refactor. Pre-existing acknowledged TODO. Does not block current functionality if batch targets healthy shards only. Should be resolved before production batch-submit load increases.

SUMMARY DASHBOARD

#	Issue	File	Severity	Decision
1	Unbuffered Signal Channel	`cmd/proxy/main.go:620`	Critical	REJECT
2	Mutating `http.DefaultClient`	`process/baseProcessor.go:75`	High	REJECT
3	DRWA DenialMessage loses context	`process/drwa_materializer.go:124`	Medium	REJECT
4	Dead constant `drwaDenialPrefix`	`process/drwa_materializer.go:36`	Medium	REJECT
5	Rate limiter missing `c.Next()`	`api/middleware/rateLimiter.go:36`	Medium	REJECT
6	Unbounded DRWA metric keys	`process/drwa_materializer.go:68`	Medium	REJECT
7	`os.Kill` is uncatchable	`cmd/proxy/main.go:621`	Medium	REJECT
8	DRWA metrics not exposed at runtime	`process/drwa_observability.go`	Low	HOLD
9	Benchmark wrong event identifier	`drwa_materializer_benchmark_test.go:29`	Low	REJECT
10	SendMultiple silent partial failure	`process/transactionProcessor.go:308`	Low	HOLD

Overall PR Status: REJECT — 8 issues require fixes before merge